Interpretable developer recommendation using contrastive learning for bug triaging

Md Jamiur Rahman Rifat
CPSC 5210
Winter 2026

Abstract

The growing dependence on software across diverse domains has resulted in a significant increase in user-reported bugs. Timely resolution of these issues is critical; however, incorrect initial developer assignment often leads to bug tossing (an act of reassigning bugs without resolution). This causes delays and reduces development efficiency. Manual assignment is both time-consuming and prone to error. Hybrid deep learning models have shown promising results in automating bug reviewer recommendation. In this work, we evaluate the effectiveness of a hybrid deep learning architecture within a contrastive learning framework. We would also like to enhance the model's transparency through explainable AI techniques such as SHAP.

1 Introduction

Software systems inevitably contain failures, faults, or errors that arise from coding discrepancies or human mistakes which are commonly termed as "bugs" [6]. Once a bug is reported, it must be triaged to determine whether it represents a valid new issue or an enhancement request. If it is valid, the report must then be assigned to a suitable developer for resolution. This takes a considerable amount of time and efforts and sometimes large-scale open-source projects such as Eclipse and Mozilla often encounter difficulties during initial bug assignment, with nearly 50% of reports reassigned because the developer originally assigned cannot resolve them effectively [7]. Given the scale of modern software development, the economic impact is also significant. In 2022, the estimated cost of finding and fixing bugs in the United States reached approximately $607 billion [4].

To address the inefficiencies in manual bug assignment, deep neural network (DNN)-based approaches have gained increasing attention. These models aim to automate the assignment process in a time-saving and effective way. However, DNNs are not without limitations. They are known to be sensitive to subtle variations in input data, which can lead to unstable or erroneous predictions. To improve robustness and representation quality, contrastive learning (CL) has emerged as a promising technique. CL focuses on learning discriminative feature representations by explicitly contrasting similar and dissimilar data samples. Although it has shown impressive results in fields such as computer vision (CV) and natural language processing (NLP), its potential for automated bug assignment has not yet been thoroughly explored [4].

Over time, researchers have proposed a variety of modeling techniques to recommend developers for bug triaging. For example, Yadav et al. [6] investigated ranking-based methods for developer recommendation. More recently, with the rise of large language models (LLMs), studies such as Chhabra et al. [2] have examined whether LLMs can effectively identify appropriate reviewers. Our motivation for integrating contrastive learning into a hybrid deep learning framework originates from the work of Geunseok et al. [7] and Rongcun et al. [4], who separately demonstrated the strengths of contrastive learning and hybrid architectures. Building on their insights, this study evaluates the effectiveness of incorporating contrastive learning into a hybrid deep learning model for bug assignment. Furthermore, we aim to enhance model transparency by employing interpretability techniques such as SHAP enabling a deeper understanding of model predictions and feature contributions in the bug assignment process.

2 Experimental Setup

2.1 Data Source

In this study, we will use BugsRepo [1], a recently published and well-curated dataset derived from Mozilla's Bugzilla ecosystem. The dataset includes 119,585 bug reports enriched with metadata and comments, along with contributor profiles for 19,351 community members, capturing their activity and collaboration history. For our model, we primarily rely on the textual content of the bug reports, specifically the summary and description as the main inputs for training and evaluation.

2.2 Pre-processing

To learn meaningful representations from bug reports, we will first apply standard text pre-processing steps to the summaries and descriptions. Specifically, we will remove punctuation, digits, non-ASCII characters, URLs, as well as embedded source code and stack traces. All text will be converted to lowercase to ensure consistency. Finally, the cleaned bug reports will be transformed into token sequences for model input.

2.3 Word Embeddings

Wang et al. (2024) [5] showed that ELMo outperforms Word2Vec and GloVe in representing bug reports due to its ability to capture contextual semantics. Motivated by this, we will use ELMo to generate 1024-dimensional contextual embeddings through its bidirectional Long-Short Term Memory (LSTM) architecture.

2.4 Recommendation Models

The process begins by extracting bug reports associated with each developer from the curated dataset, followed by pre-processing to clean and standardize the data. The data is then converted into vectors using ELMo which is then fed into our deep learning model. To generate the final output as a probability distribution over suitable developers, a softmax layer is used in the final classification layer. The key modification in the proposed architecture is the integration of supervised contrastive learning loss alongside the traditional

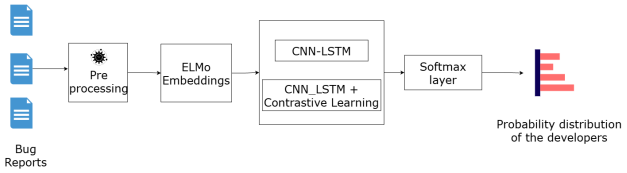

Figure 1: Overall framework of the proposed approach.

cross-entropy loss. While the softmax layer produces developer probabilities, the supervised contrastive loss is applied to the embedding space to enhance representation separability and improve discriminative learning.

In this architecture, the CNN component captures local and spatial patterns in the data, while the LSTM component models sequential and temporal dependencies within bug reports. By integrating both, the model generates accurate developer recommendations. Our proposed framework is illustrated in (Fig. 1).

2.4.1 Contrastive learning. Contrastive learning is a representation learning technique that improves model performance by bringing similar samples (positive pairs) closer in the embedding space while pushing dissimilar samples (negative pairs) farther apart. Originally popularized in self-supervised learning, van den Oord et al. [3] introduced the InfoNCE loss to learn meaningful representations from high-dimensional data. In our setting, bug reports resolved by the same developer will be treated as positive samples, whereas bug reports assigned to different developers will be considered negative samples.

2.4.2 Hybrid CNN-LSTM base Model. To generate personalized developer recommendations for new bug reports, we employ a hybrid CNN-LSTM architecture. The Convolutional Neural Network (CNN) component processes high-dimensional textual features and captures local semantic patterns through convolutional layers, with padding applied to preserve spatial dimensions. Max pooling is used to retain the most informative features while reducing computational complexity.

The extracted features will be then fed into an LSTM layer, which models sequential dependencies and long-term contextual information within the bug reports. By combining CNN's ability to learn local patterns with LSTM's strength in capturing temporal relationships, the model effectively produces accurate developer recommendations.

2.4.3 Hybrid CNN-LSTM Model with contrastive loss. We will use contrastive loss to learn more discriminative developer embeddings. Bug reports resolved by the same developer are treated as positive pairs, while those handled by different developers form negative pairs. The model is trained to bring positive pairs closer and push negative pairs farther apart in the embedding space.

3 Evaluation Metrics

To evaluate the performance of the proposed model, we will use standard machine learning metrics [7], namely precision, recall, F1-score, and accuracy. These metrics assess different aspects of the recommendation quality.

$$\text{Precision} = \frac{TP}{TP + FP} \tag{1}$$

Precision measures the proportion of correctly recommended developers among all predicted positive recommendations.

$$\text{Recall} = \frac{TP}{TP + FN} \tag{2}$$

Recall evaluates the model's ability to identify all relevant developers for bug reports.

$$\text{F1-score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3}$$

The F1-score represents the harmonic mean of precision and recall, providing a balanced evaluation metric, especially for imbalanced datasets.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{4}$$

Accuracy measures the overall proportion of correct predictions among all predictions made by the model.

4 Future Directions

Bug reports also include attributes such as bug type and severity, and reviewer profiles capture expertise and past activity. Prior studies show that reviewer expertise strongly affects recommendation performance [6][7]. Therefore, incorporating these features may further improve the model's effectiveness.

References

[1] Jagrit Acharya and Gouri Ginde. 2025. BugsRepo: A Comprehensive Curated Dataset of Bug Reports, Comments and Contributors Information from Bugzilla. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE '25)*. Association for Computing Machinery, New York, NY, USA, 986–993. doi:10.1145/3756681.3756994

[2] Deepshikha Chhabra and Raman Chadha. 2024. Automatic Bug Triaging Process: An Enhanced Machine Learning Approach through Large Language Models. *Engineering, Technology & Applied Science Research* 14, 6 (Dec. 2024), 18557–18562. doi:10.48084/etasr.8829

[3] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2019. Representation Learning with Contrastive Predictive Coding. doi:10.48550/arXiv.1807.03748 arXiv:1807.03748 [cs].

[4] Rongcun Wang, Xingyu Ji, Yuan Tian, Senlei Xu, Xiaobing Sun, and Shujuan Jiang. 2025. Fixer-level supervised contrastive learning for bug assignment. *Empirical Software Engineering* 30, 3 (March 2025), 76. doi:10.1007/s10664-025-10634-0

[5] Rongcun Wang, Xingyu Ji, Senlei Xu, Yuan Tian, Shujuan Jiang, and Rubing Huang. 2024. An empirical assessment of different word embedding and deep learning models for bug assignment. *Journal of Systems and Software* 210 (April 2024), 111961. doi:10.1016/j.jss.2024.111961

[6] Asmita Yadav, Sandeep Kumar Singh, and Jasjit S. Suri. 2019. Ranking of software developers based on expertise score for bug triaging. *Information and Software Technology* 112 (Aug. 2019), 1–17. doi:10.1016/j.infsof.2019.03.014

[7] Geunseok Yang, Jinfeng Ji, and Dongkyu Kim. 2025. Enhancing Bug Assignment with Developer-Specific Feature Extraction and Hybrid Deep Learning. *Electronics* 14, 12 (June 2025). doi:10.3390/electronics14122493